

Internet das coisas

Segurança no Desenvolvimento de Software

XSS (Cross-Site Scripting)

Prof. Ana Beatriz Morais

SET/2026

O que veremos hoje

- O que é XSS
- Como o atacante injeta o Javascript
- Tipos de XSS:
 - Refletido
 - Armazenado
- Como evitar cross-site scripting
- Atividade prática

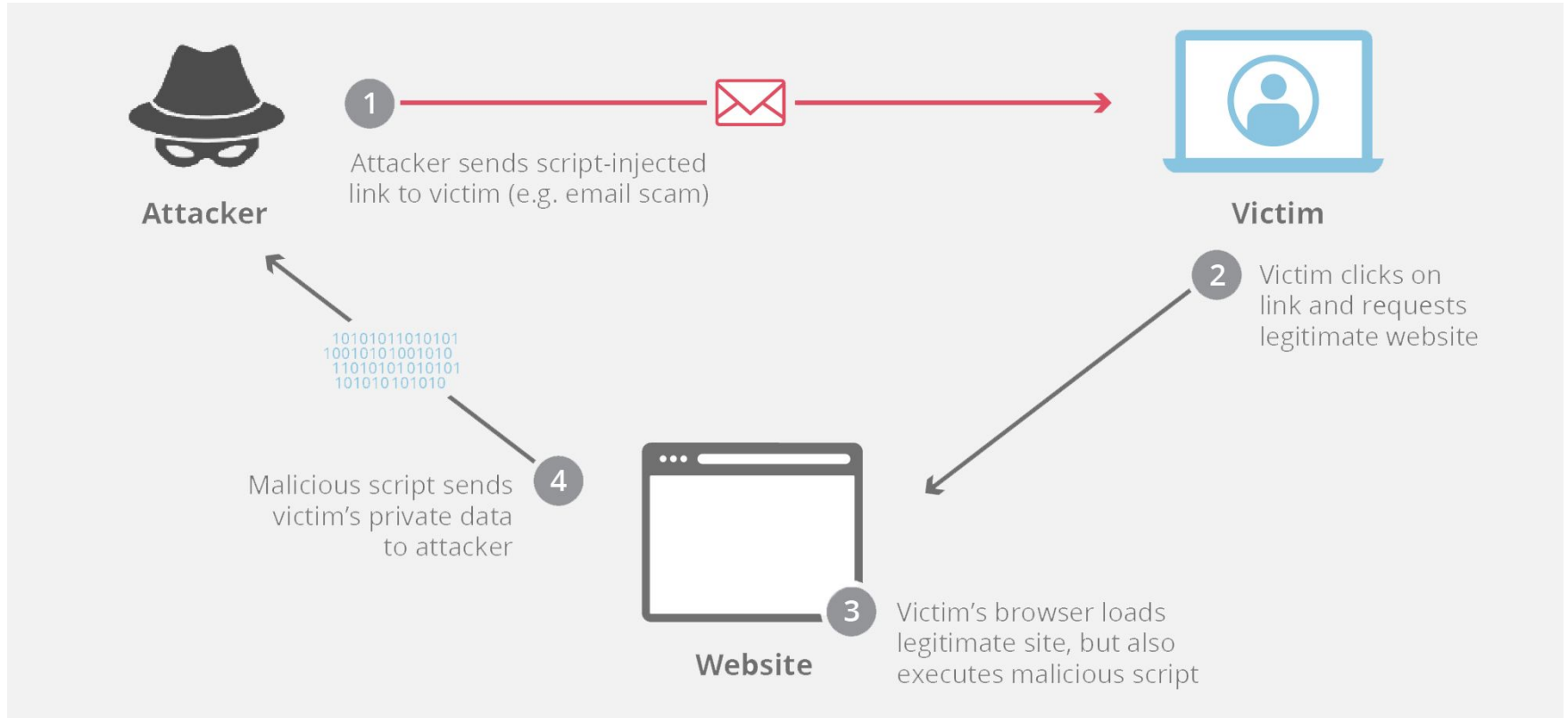


XSS (Cross-Site Scripting)

XSS (Cross-Site Scripting)

- **O que significa?**
 - É uma **falha de segurança em aplicações web** que **permite a um invasor injetar códigos maliciosos (geralmente em JavaScript)** em páginas visualizadas por outros usuários.
 - Em termos técnicos:
 - É um ataque de injeção de código do lado do cliente.

XSS (Cross-Site Scripting)



Código do lado do cliente

- **O que significa?**

- É o código JavaScript que é executado na máquina de um usuário.
- Em termos de sites, o código do lado do cliente geralmente é um código executado pelo navegador web depois que o navegador carrega uma página da web.

Como o atacante injeta o Javascript?

- Um exemplo útil de ataques de cross-site scripting é comumente **visto em sites que possuem fóruns de comentários não validados.**
- Invasor posta um comentário que consiste em código executável envolto das tags “<script></script>”.
- Essas tags informam a um navegador web que interprete tudo entre as tags como código JavaScript.
- Assim que esse comentário estiver na página, quando qualquer outro usuário carregar esse site, o código malicioso entre as tags de script será executado pelo navegador web e o usuário se tornará vítima do ataque.

Exemplo de Cross-Site Scripting (XSS)

Um invasor publica um comentário com código JavaScript. Quando outros usuários acessam a página, o código é executado no navegador deles.

1 O invasor publica um comentário malicioso



Invasor

https://meusite.com/forum

Fórum de Discussão

Novo comentário

`<script>alert('Você foi vítima de XSS!');</script>`

Publicar comentário

O invasor digita um comentário contendo código JavaScript entre as tags `<script></script>`.

2 O comentário é salvo e exibido no fórum

https://meusite.com/forum

Fórum de Discussão

João
Ótimo conteúdo!

Invasor
`<script>alert('Você foi vítima de XSS!');</script>`

Maria
Muito útil, obrigado!

O sistema exibe o comentário do invasor na página, sem validar ou remover o código.

3 Outro usuário acessa a página



Outro usuário

https://meusite.com/forum

Fórum de Discussão

João
Ótimo conteúdo!

Invasor
`<script>alert('Você foi vítima de XSS!');</script>`

Maria
Muito útil, obrigado!

Quando outro usuário carrega o fórum, o navegador interpreta o código dentro das tags `<script>` e o executa.

4 O código é executado no navegador da vítima

https://meusite.com/forum

Fórum de Discussão

João
Ótimo conteúdo!

Invasor
`<script>alert('Você foi vítima de XSS!');</script>`

Maria
Muito útil, obrigado!

meusite.com diz
Você foi vítima de XSS!
OK

O JavaScript é executado no navegador do usuário, e ele se torna vítima do ataque.



Esse é apenas um exemplo simples.

Em ataques reais, o código pode ser muito mais perigoso, como roubar cookies de sessão, redirecionar o usuário, alterar o conteúdo da página ou realizar ações em nome da vítima.

Por que um ataque XSS pode ser perigoso?

- Os ataques de Cross-Site Scripting (XSS) são perigosos porque permitem que um invasor execute JavaScript no navegador da vítima, dentro do contexto de uma aplicação confiável.
- Dependendo das proteções do navegador e da aplicação, esse código pode ter acesso a informações e funcionalidades disponíveis naquela página.

Por que um ataque XSS pode ser perigoso?

- **O que um atacante pode tentar fazer através de um XSS?**
 - Roubar informações de sessão: alguns cookies podem ser acessados pelo JavaScript. Caso um cookie de sessão esteja acessível, um atacante pode tentar capturá-lo e utilizá-lo para se passar pela vítima. Cookies configurados como HttpOnly, porém, não podem ser lidos diretamente pelo JavaScript.
 - Realizar requisições em nome da vítima: o JavaScript pode fazer requisições HTTP para a própria aplicação. Como o código está sendo executado no navegador de um usuário autenticado, isso pode permitir a realização de ações utilizando a sessão da vítima, dependendo das proteções existentes.

Por que um ataque XSS pode ser perigoso?

- **O que um atacante pode tentar fazer através de um XSS?**
 - Enviar informações para servidores externos: um código malicioso pode tentar coletar informações acessíveis à página e enviá-las para um servidor controlado pelo atacante.
 - Modificar o conteúdo da página: o atacante pode alterar elementos da interface, apresentar mensagens falsas ou até criar formulários falsos para tentar capturar informações digitadas pelo usuário.
 - Tentar acessar recursos do navegador: aplicações web podem utilizar APIs relacionadas a recursos como localização, câmera e microfone. Entretanto, o acesso a vários desses recursos depende das permissões concedidas pelo usuário e das políticas de segurança do navegador.

Ou só abusar da
concatenação
de string no JS

```
("text" + alert())
```

Tipos de XSS – Refletido e Armazenado

Tipos de XSS

Mais populares:

Tipo	Onde fica o conteúdo?	Quem pode ser afetado?
Refletido	Normalmente vem da requisição/URL	Quem acessa aquela requisição/link
Armazenado/Persistente	Fica salvo no sistema	Vários usuários

XSS – Refletido

Cross-site scripting refletido

- Com um **ataque refletido**, um **código malicioso é adicionado ao final do URL de um site**; muitas vezes este será um site legítimo e confiável.
- Quando a vítima carrega este link em seu navegador web, o navegador executará o código injetado no URL.
- O **invasor geralmente usa** alguma forma de **engenharia social para induzir a vítima a clicar no link**.

Cross-site scripting refletido

- Exemplo:
 - Um usuário pode receber um e-mail de aparência legítima que afirma vir de seu banco.
 - O e-mail solicitará que ele adote alguma ação no site do banco e fornece um link. O link pode ser assim:
 - **`http://bancodobrasil.com/index.php?user=<script>here is some bad code!</script>`**
 - **Embora a primeira parte do URL pareça seguro e contenha o domínio de um site confiável, o código injetado no final do URL pode ser malicioso.**

Cross-site scripting refletido

Exemplo de XSS via link em e-mail

Um usuário recebe um e-mail aparentemente legítimo, mas o link contém código malicioso.

O domínio parece confiável, mas o código injetado no final do URL pode ser malicioso.

1 Usuário recebe um e-mail falso



2 O e-mail contém um link com código malicioso

O link fornecido pode ser assim:

`http://bancodobrasil.com/index.php?user=<script>here is some bad code!</script>`

Domínio confiável
(aparenta ser seguro)

Código injetado
(pode ser malicioso)

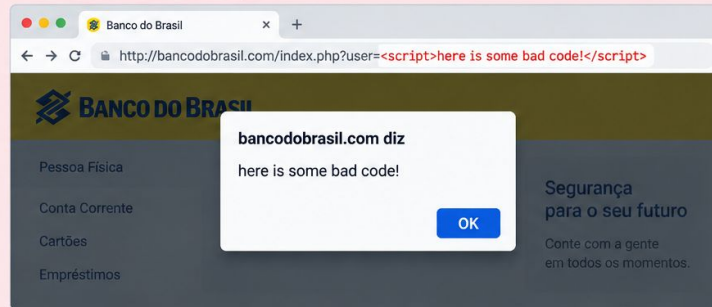
3 O usuário clica no link

O usuário acredita que é um link legítimo e é direcionado para o site.



4 O código é executado no navegador

O site é carregado e o navegador interpreta o código injetado, executando o JavaScript.



XSS – Armazenado/Persistente

`<script>`

`</script>`

Vulnerability

Scripting (XSS)

`<script>alert("XSS")</script>`

OWASP

Identifying: Script Cross Site Scripting (XSS)

Alert

152.158.1.1

Cross-site scripting persistente

- É o mesmo caso do fórum de comentários.
- **Se o site não validar adequadamente as entradas** para conteúdo gerado pelo usuário, **um invasor poderá inserir um código que os navegadores de outros usuários executarão quando a página for carregada.**
- Outro exemplo é uma rede social onde cada usuário pode editar sua biografia. O sistema salva essa biografia no banco de dados e depois a apresenta no perfil.
- Um atacante percebe que o campo não é tratado corretamente e, em um ambiente de laboratório, cadastra algo como:
 - ****

Cross-site scripting persistente

Exemplo de XSS Persistente (Armazenado)

O código malicioso é salvo no sistema e será executado sempre que o perfil for acessado.

1 O atacante edita sua biografia

O atacante acessa seu perfil e insere um código malicioso no campo "Biografia".

SocialNet

Buscar pessoas...

Inicio

Meu perfil

Amigos

Mensagens

Configurações

Editar perfil

Nome

Usuário Malicioso

Biografia

Salvar perfil

2 O sistema salva no banco de dados

O conteúdo não é validado e é salvo exatamente como foi inserido.

Tabela: usuarios

id	nome	biografia
1	Usuário Malicioso	

3 O perfil é exibido para outros usuários

Quando outra pessoa acessa o perfil do atacante, o sistema recupera a biografia do banco de dados e a exibe na página.

SocialNet

Buscar pessoas...

Usuário Malicioso

@usuario_malicioso

10 publicações 250 seguidores 30 seguindo

Sobre mim

4 O JavaScript é executado no navegador da vítima

O navegador interpreta o código e executa o JavaScript.

SocialNet

Buscar pessoas...

Usuário Malicioso

@usuario_malicioso

10 publicações 250 seguidores 30 seguindo

Sobre mim

socialnet.com diz XSS

OK

5 O ataque continua ativo

O código permanece salvo no sistema e será executado sempre que qualquer usuário acessar o perfil, até que seja removido.



Isso é um XSS persistente

Porque o conteúdo malicioso fica armazenado no sistema, afetando todos que acessarem aquela página.

Como evitar o XSS ?

Como evitar cross-site scripting ?

- Não existe uma estratégia única para mitigar cross-site scripting e diferentes tipos de aplicativos web exigem diferentes níveis de proteção.
- Uma série de medidas de proteção podem ser tomadas.
 - **Se possível, evitar HTML nas entradas:**
 - impedir que os usuários postem HTML em entradas de formulário.
Existem outras opções que permitem aos usuários criar conteúdo rico sem o uso de HTML, como editores de markdown e WYSIWYG
 - **Validar as entradas:**
 - Validação significa implementar regras que evitem que um usuário poste dados em um formulário que não atendem a determinados critérios

Como evitar cross-site scripting ?

- **Adotar medidas de segurança de cookies**

- Os aplicativos web também podem definir regras especiais para o manuseio de cookies que podem mitigar o roubo de cookies por meio de ataques cross-site scripting. Os cookies podem ser vinculados a endereços de IP específicos para que os invasores de cross-site scripting não possam acessar. Além disso, podem ser criadas regras para bloquear completamente o JavaScript de acessar os cookies.

- **Higienização de dados**

- A higienização de dados é semelhante à validação, mas ocorre após os dados já terem sido postados no servidor web, mas ainda antes de serem exibidos para outro usuário. Existem várias ferramentas on-line que podem higienizar o HTML e filtrar quaisquer injeções de código malicioso.

Como evitar cross-site scripting ?

- **Outros exemplos de coisas que você não deve fazer com entrada do usuário:**
 - `element.innerHTML = entradaUsuario;`
 - `element.outerHTML = entradaUsuario;`
 - `document.write(entradaUsuario);`
 - `document.writeln(entradaUsuario);`
 - Evitar construir HTML assim:
 - `lista.innerHTML += ` <p>${comentario}</p>`;`
 - `element.insertAdjacentHTML( "beforeend", entradaUsuario);`



XSS ATTACK in 20 Second



Server



Learn Cyber Security in
a fun way

Server



Atividade Prática

Atividade

- Faça o download da pasta “aula_09-09” para o seu computador. Nela existem duas aplicações propositalmente vulneráveis. O objetivo da atividade é identificar, explorar em ambiente controlado e corrigir as vulnerabilidades XSS, sem comprometer o funcionamento normal das aplicações.
- **Parte 1 – XSS Refletido (index.html da raiz)**
 - Abra e execute o arquivo index.html localizado na raiz da pasta.
 - Faça testes inserindo textos comuns e tags HTML nos campos disponíveis.
 - Verifique se o navegador interpreta o conteúdo inserido como HTML.
 - Identifique qual trecho do código permite esse comportamento.
 - Explique por que ele representa uma vulnerabilidade.
 - Corrija o código para que a entrada do usuário seja exibida apenas como texto, sem impedir o funcionamento normal da página.
 - Após a correção, repita os testes e confirme que as tags HTML não são mais interpretadas.

Atividade

- **Parte 2 – XSS Armazenado (pasta back-end)**

- Execute a aplicação localizada na pasta back-end.
- Publique comentários normais e verifique o funcionamento da aplicação.
- Teste a inserção de conteúdo HTML no campo de comentário.
- Analise o código e identifique onde está a vulnerabilidade que permite que o conteúdo armazenado seja interpretado pelo navegador.
- Corrija a aplicação sem impedir que os usuários publiquem comentários normais.
- Após a correção, repita os testes e confirme que conteúdos potencialmente maliciosos são tratados como dados/texto e não como código executável.
- **Entrega:** apresente o código vulnerável identificado, explique por que ele era vulnerável, mostre a alteração realizada e demonstre que, após a correção, comentários normais continuam funcionando enquanto HTML/JavaScript inserido pelo usuário não é executado.
- **Importante:** não basta fazer o ataque parar de funcionar. Vocês devem conseguir explicar por que funcionava antes e por que a solução implementada impede o XSS.

Atividade

- **Enviar a atividade corrigida, armazenada no github e me enviar o link no meu email:**
 - **ana.morais@edu.sc.senai.br**
- **Data/hora de entrega:**
 - Até o dia 11/09, com horário limite até 19h.



Serviço Nacional de Aprendizagem Industrial

PELO FUTURO DO TRABALHO

0800 048 1212     **sc.senai.br**

Rodovia Admar Gonzaga, 2765 - Itacorubi - 88034-001 - Florianópolis, SC